# 테스트 절차서

적심·적과 농작업 로봇 관제 시각화 시뮬레이터

# 목차

**1. 테스트 개요**

1.1 본 문서의 목적

1.2 테스트 대상

1.3 테스트 역할

1.4 테스트 흐름

**2. 테스트 환경 사양**

2.1 운영체제

2.2 브라우저

2.3 GPU

2.4 네트워크

2.5 Node 환경 (소스 빌드 테스트 시)

**3. 테스트 절차 (5단계)**

3.1 단계 1 — 빌드 테스트

3.2 단계 2 — 실행 테스트

3.3 단계 3 — 시나리오 구현 테스트

3.4 단계 4 — 카메라 스트리밍 / 디텍션 테스트

3.5 단계 5 — 매뉴얼 테스트

**4. 자료별 평가 기준**

4.1 시뮬레이터 실행 빌드

4.2 소스코드 패키지

4.3 디텍션 모델 자산

4.4 시나리오·데이터셋

4.5 시스템 아키텍처 설계서

4.6 운용 매뉴얼

4.7 개발자/관리자 가이드

4.8 테스트 절차서 (본 문서)

4.9 시험 결과 보고서

4.10 운용 데모 녹화

**5. 시험 도구**

5.1 자동화 도구 (Playwright probe)

5.2 자동화 실행 명령

5.3 수동 체크리스트

**6. 테스트 결과 기록 양식**

6.1 항목별 결과 표

6.2 종합 판정 표

6.3 종합 결과

**7. 보완 처리 절차**

7.1 보완 요청

7.2 보완 자료 제출

7.3 재테스트

7.4 최종 판정

**8. 입회 절차**

8.1 입회자

8.2 입회 기록

8.3 입회 결과 문서

**9. 부록 — 자동화 probe 실행 명령**

9.1 환경 준비

9.2 dev server 실행

9.3 probe 실행

9.4 결과 정리

9.5 종료

# 1. 테스트 개요

## 1.1 본 문서의 목적

본 문서는 적심·적과 농작업 로봇 관제 시각화 시뮬레이터가 정상 동작·정확한 생육 모델·재현 가능한 디텍션·재현 가능한 시나리오라는 4 영역의 품질 기준을 충족하는지 확인하기 위한 테스트 절차를 정의한다. 본 문서를 따라 테스트를 진행하면 시뮬레이터의 모든 핵심 기능이 통과/실패로 평가된다.

## 1.2 테스트 대상

본 문서가 다루는 테스트 대상은 동일 패키지에 포함된 다음 10 종의 자료다.

| # | 자료 | 파일 |
|---|---|---|
| 1 | 시뮬레이터 실행 빌드 | `02_Monitoring_Sim_Build.zip` |
| 2 | 소스코드 패키지 | `03_Source_Code.zip` |
| 3 | 디텍션 모델 자산 | `04_Detection_Models.zip` |
| 4 | 시나리오·데이터셋 | `05_Scenario_Datasets.zip` |
| 5 | 시스템 아키텍처 설계서 | `06_Architecture_Spec.pdf` |
| 6 | 운용 매뉴얼 | `07_User_Manual.pdf` |
| 7 | 개발자/관리자 가이드 | `08_Developer_Guide.pdf` |
| 8 | 본 테스트 절차서 | `09_Inspection_Plan.pdf` |
| 9 | 시험 결과 보고서 | `10_Test_Report.pdf` |
| 10 | 운용 데모 녹화 | `11_Demo_Recording.zip` |

## 1.3 테스트 역할

| 역할 | 책임 |
|---|---|
| 테스트 진행자 | 절차 수행, 결과 기록, 시험 결과 보고서 작성 |
| 참관자 | 결과 확인, 보완사항 판단 |
| 시뮬레이터 제공자 | 보완 자료 대응 |

## 1.4 테스트 흐름

본 절차서는 다음 5 단계로 구성된다.

1. 환경 사양 점검 (§2)
2. 단계별 테스트 절차 (§3)
3. 자료별 평가 기준 (§4)
4. 자동/수동 시험 도구 (§5)
5. 결과 기록 양식 (§6)

본 절차서는 향후 보완 테스트 또는 재테스트 진행 시에도 동일한 형태로 재사용된다.

# 2. 테스트 환경 사양

테스트는 사양이 보장된 환경에서 수행한다. 사용자가 다음 환경을 준비한다.

## 2.1 운영체제

| OS | 버전 | 비고 |
|---|---|---|
| Windows 11 | 22H2 이상 | 권장 |
| macOS 14 (Sonoma) | 이상 | 본 테스트에 사용 |
| Ubuntu 22.04 LTS | 이상 | 가능 |

## 2.2 브라우저

| 브라우저 | 최소 버전 |
|---|---|
| Google Chrome | 120 이상 |
| Mozilla Firefox | 120 이상 |

브라우저는 WebGL2 활성 상태여야 한다.

## 2.3 GPU

| 등급 | 사양 | fps 기대 |
|---|---|---|
| 최소 | Intel Iris Xe 또는 동등 | 30 fps |
| 권장 | NVIDIA RTX 3060 또는 동등 이상 | 60 fps |

## 2.4 네트워크

본 테스트는 **localhost 전용**이다. 외부망 의존 없음.

WS endpoint: `ws://localhost:8090/robot/phenotyping/gimbal`

## 2.5 Node 환경 (소스 빌드 테스트 시)

| 도구 | 버전 |
|---|---|
| Node.js | 20 LTS |
| npm | 10 이상 |
| Git | 2.40 이상 |

# 3. 테스트 절차 (5단계)

## 3.1 단계 1 — 빌드 테스트

**목적**: 시뮬레이터 실행 빌드( `02_Monitoring_Sim_Build.zip` )의 무결성·실행 확인.

**절차**:

1. zip 파일의 무결성 확인 ( `unzip -t` )
2. 임의의 폴더에 압축 해제
3. 정적 서버 띄우기 ( `npx serve <폴더>` 또는 `python3 -m http.server` )
4. 브라우저로 진입
5. EntryScreen 정상 표시 확인
6. 첫 프레임 렌더 시간 측정 (브라우저 DevTools Performance)

**평가 기준**:

- zip 무결성 통과
- 부팅 무오류 (Console 에러 0건 또는 무관 warning만)
- 권장 사양 기준 첫 프레임 ≤ 5초, 최소 사양 ≤ 20초

## 3.2 단계 2 — 실행 테스트

**목적**: 시나리오 picker → Workbench 진입 → 시뮬레이터 정상 표시 확인.

**절차**:

1. EntryScreen에서 Workbench 모드 선택
2. 시나리오 Picker에서 `phenotyping-D110-survey` 선택
3. Workbench 진입 후 다음 확인:

- 메인 카메라 뷰 정상 표시
- PiP ( `GIMBAL CAM` ) 표시
- TaskPanel 활성 (START SURVEY 버튼 등)

4. fps 측정 (DevTools 또는 PerfHUD)

**평가 기준**:

- 모든 시나리오 항목 활성
- 진입 후 fps ≥ 30 (최소) 또는 60 (권장)
- UI 요소 누락 0건

## 3.3 단계 3 — 시나리오 구현 테스트

**목적**: 적심·적과 관제 시나리오 3종이 완주 가능한지 확인.

**시나리오 3종**:

1. **좌우 베드 페노타이핑 서베이** — START SURVEY 후 완료까지
2. **짐벌 카메라 PiP** — 스트림 모드와 in-canvas 모드 토글 시 화면 동일성
3. **결과 시트** — 서베이 완료 후 자동 갱신, bin별 카운트 표시

**절차**:

각 시나리오를 매뉴얼( `07_User_Manual.pdf` ) 절차에 따라 수행하고, 다음 시점에 스크린샷을 캡처:

- 진입 직후
- 서베이 50% 진행 시점
- 서베이 완료 직후
- 결과 시트 표시

**평가 기준**:

- 각 시나리오 완주까지 무오류
- 결과 시트의 카운트 합계 > 0
- 두 PiP 모드의 출력 동일성 시각 확인

## 3.4 단계 4 — 카메라 스트리밍 / 디텍션 테스트

**목적**: WS 스트림 producer·consumer 연결 및 디텍션 결과 매핑 확인.

**절차**:

1. DevTools Network 탭 WS 필터
2. 시나리오 진입 후 `/robot/phenotyping/gimbal?role=producer` 와 `?role=consumer` 두 WS 연결 확인
3. 연결 시간 측정 (open 까지 < 5초)
4. PiP에 색깔 원 오버레이가 그려지는지 확인
5. Network 탭에서 임의로 ws producer 연결 끊고 자동 재연결 확인
6. Playwright probe `_probe-stream-pipeline.spec.ts` 실행

**평가 기준**:

- WS 연결 ≤ 5초
- 디텍션 오버레이가 식물 위에 표시
- 디텍션 결과의 3D 월드 좌표 역투영 동작 (raycast hit이 plant mesh에 들어감)
- Disconnect 시 자동 재연결 동작

## 3.5 단계 5 — 매뉴얼 테스트

**목적**: 신규 사용자가 운용 매뉴얼(`07_User_Manual.pdf`)만 보고 ①~④를 재현 가능한지 확인.

**절차**:

1. 매뉴얼을 처음 보는 독자가 매뉴얼만 가지고 단계 1~4를 다시 수행
2. 각 단계의 빠진 내용·모호한 표현·추가 설명 필요 사항 기록

**평가 기준**:

- 매뉴얼만으로 단계 1~4 재현 성공
- 빠진 단계 0건
- 추가 설명 필요 사항이 있다면 보완 권고로 기록 (보완 시 PASS 변경)

# 4. 자료별 평가 기준

각 자료(#1~#10)의 평가 기준은 input JSON `inspection_criteria`와 1:1 매핑된다. 본 절은 그것을 정량/정성 기준으로 분해해서 명시한다.

## 4.1 시뮬레이터 실행 빌드

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| zip 무결성 | `unzip -t` 통과 | CLI |
| 부팅 무오류 | Console 에러 0건 | DevTools Console |
| 첫 프레임 렌더 시간 | ≤ 5s (권장) | DevTools Performance |
| 권장 사양 fps | ≥ 60 fps | PerfHUD |
| 최소 사양 fps | ≥ 30 fps | PerfHUD |

## 4.2 소스코드 패키지

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| `package-lock.json` 포함 | 존재 | `unzip -l \| grep package-lock` |
| `npm install` 무오류 | `npm install` 성공 | CLI |
| `npm run build` 무오류 | exit code 0 | CLI |
| `npx tsc --noEmit` | exit code 0 | CLI |

## 4.3 디텍션 모델 자산

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| ONNX 파일 SHA-256 일치 | yolov8n.onnx.sha256 값과 일치 | `shasum -a 256` |
| 입출력 텐서 사양 문서화 | `model_metadata.json` 존재 | 파일 확인 |
| 클래스 매핑 명시 | `model_metadata.json.class_mapping` 존재 | 파일 확인 |
| HSV 파라미터 명시 | `hsv_parameters.json` 존재 | 파일 확인 |
| 추론 결과 캡처 | `sample_inference.png` 존재 | 파일 확인 |

## 4.4 시나리오·데이터셋

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 시드 명시 | `showcaseSeed.ts` 존재 | 파일 확인 |
| 시나리오 카탈로그 | `scenarios/loader.ts` 존재 | 파일 확인 |
| 시드별 재현 일관성 | 같은 시드 → 같은 결과 | 두 번 실행하여 비교 |
| 토마토 모델 SSOT | `tomato-engine-models/INDEX.jsonc` 존재 | 파일 확인 |

## 4.5 시스템 아키텍처 설계서

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 모듈 계층도 포함 | 본문 §3 또는 §모듈 계층 | 본문 확인 |
| 좌표계 명세 | 본문 §좌표계 규약 | 본문 확인 |
| Mesh Anchor 명세 | 본문 §Mesh Anchor 규약 | 본문 확인 |
| 스트림 프로토콜 명세 | 본문 §스트림 프로토콜 | 본문 확인 |
| 로깅 명세 | 본문 §로깅 시스템 | 본문 확인 |
| README와 일치성 | README의 기술 스택과 본문 §기술 스택 동일 | 비교 |

## 4.6 운용 매뉴얼

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 설치 절차 | 본문 §4 | 본문 확인 |
| 실행 절차 | 본문 §5 | 본문 확인 |
| 시나리오 운용 절차 | 본문 §6, §7 | 본문 확인 |
| 결과 해석 | 본문 §8, §9 | 본문 확인 |
| 트러블슈팅 | 본문 §13 | 본문 확인 |
| 신규 사용자 단독 실행 가능 | 매뉴얼만으로 시연 성공 | 테스트 단계 5 |

## 4.7 개발자/관리자 가이드

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 빌드 명령 | 본문 §4 | 본문 확인 |
| 시나리오 추가 절차 | 본문 §5 | 본문 확인 |
| 디텍터 swap 절차 | 본문 §6 | 본문 확인 |
| 디버그 옵션 명시 | 본문 §10 | 본문 확인 |
| 모델 업데이트 절차 | 본문 §14 | 본문 확인 |

## 4.8 테스트 절차서 (본 문서)

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 테스트 환경 명시 | §2 | 본문 확인 |
| 테스트 절차 명시 | §3 (5단계) | 본문 확인 |
| 평가 기준 명시 | §4 (10건) | 본문 확인 |
| 시험 도구 명시 | §5 | 본문 확인 |

## 4.9 시험 결과 보고서

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 시험 환경 명시 | 본문 §2 | 본문 확인 |
| #1~#10 모두 결과 기록 | 본문 §5 | 본문 확인 |
| 자동화 probe 출력 포함 | 본문 §6 | 본문 확인 |
| 판정 종합 | 본문 §7 | 본문 확인 |

## 4.10 운용 데모 녹화

| 항목 | 기준 | 측정 방법 |
|---|---|---|
| 시연 시나리오 포함 | 영상 내용 확인 | 영상 재생 |
| 화면·음성 누락 0건 | — | 영상 재생 |
| 해상도 ≥ 1280×720 | — | 메타데이터 확인 |
| 포맷 mp4 또는 webm | — | 메타데이터 확인 |

# 5. 시험 도구

## 5.1 자동화 도구 (Playwright probe)

| 도구 | 경로 | 용도 |
|---|---|---|
| 스트리밍 파이프라인 | `tests/architecture/_probe-stream-pipeline.spec.ts` | WS producer 연결 + 바이너리 프레임 ≥ 1 도착 |
| 디텍션 흐름 | `tests/architecture/_probe-detect-flow.spec.ts` | survey → detect 흐름 |
| 짐벌 추적 | `tests/architecture/_probe-gimbal-tracking.spec.ts` | gimbal cam 추적 |
| 페노타이핑 부팅 | `tests/architecture/_probe-phenotyping.spec.ts` | crash + console 검사 |
| 로깅 enforcement | `tests/architecture/logger-enforcement.spec.ts` | `console.*` 직접 호출 0건 |

## 5.2 자동화 실행 명령

```
npx playwright test tests/architecture/_probe-stream-pipeline.spec.ts \
 tests/architecture/_probe-detect-flow.spec.ts \
 tests/architecture/_probe-gimbal-tracking.spec.ts \
 tests/architecture/_probe-phenotyping.spec.ts \
 tests/architecture/logger-enforcement.spec.ts
```

## 5.3 수동 체크리스트

매뉴얼·운용 시연·문서 일치성 등은 수동 검사. 본 plan 의 §3 절차를 단계별로 따라가며 통과/실패/관찰필요 판정.

# 6. 테스트 결과 기록 양식

테스트 결과는 다음 표 형식으로 기록한다.

## 6.1 항목별 결과 표

| 항목 | 시험 도구 | 시험 일시 | 결과 | 관찰사항 |
|---|---|---|---|---|
| (자동 채움) | (자동 채움) | YYYY.MM.DD HH:MM | 통과 / 보완 / 관찰필요 | 자유 기술 |

## 6.2 종합 판정 표

| 자료 | 결과 | 테스트 의견 |
|---|---|---|
| #1 시뮬레이터 실행 빌드 | 통과 / 보완 / 관찰필요 | — |
| #2 소스코드 패키지 | 통과 / 보완 / 관찰필요 | — |
| #3 디텍션 모델 자산 | 통과 / 보완 / 관찰필요 | — |
| #4 시나리오·데이터셋 | 통과 / 보완 / 관찰필요 | — |
| #5 시스템 아키텍처 설계서 | 통과 / 보완 / 관찰필요 | — |
| #6 운용 매뉴얼 | 통과 / 보완 / 관찰필요 | — |
| #7 개발자/관리자 가이드 | 통과 / 보완 / 관찰필요 | — |
| #8 테스트 절차서 | 통과 / 보완 / 관찰필요 | — |
| #9 시험 결과 보고서 | 통과 / 보완 / 관찰필요 | — |
| #10 운용 데모 녹화 | 통과 / 보완 / 관찰필요 | — |

## 6.3 종합 결과

본 plan 의 모든 항목이 **통과**으로 판정되면 테스트 완료. 보완 또는 관찰필요 항목이 있으면 §7 보완 처리 절차로 진행.

# 7. 보완 처리 절차

## 7.1 보완 요청

독자는 보완 사항을 정리하여 제공자에 보완 요청 문서를 발송한다. 보완 요청 문서에는 다음을 포함한다.

- 보완 항목
- 보완 기준
- 보완 기한
- 보완 자료 제출 양식

## 7.2 보완 자료 제출

제공자는 보완 기한 내에 보완 자료를 제출한다.

## 7.3 재테스트

독자는 보완 자료를 받아 §3 절차에 따라 재테스트한다.

## 7.4 최종 판정

재테스트 결과 적합으로 판정되면 테스트 완료. 여전히 실패이면 추가 보완 요청.

# 8. 입회 절차

## 8.1 입회자

| 역할 | 소속 |
|---|---|
| 테스트 진행자 | |
| 테스트 입회자 (책임) | |
| 자료 제공자 | 제공자 |

## 8.2 입회 기록

| 항목 | 내용 |
|---|---|
| 시험 일시 | YYYY.MM.DD HH:MM |
| 테스트 장소 | ( 내부 또는 협의 장소) |
| 테스트 진행자 | 성명·서명 |
| 테스트 입회자 | 성명·서명 |
| 자료 제공자 | 성명·서명 |
| 테스트 결과 | 통과 / 보완 / 관찰필요 |
| 보완사항 | 자유 기술 |

## 8.3 입회 결과 문서

입회 완료 후 통과 확인서(관련 문서)에 결과를 정리.

# 9. 부록 — 자동화 probe 실행 명령

## 9.1 환경 준비

```
git clone <repo-url> MonitoringSim
cd MonitoringSim
npm install
```

## 9.2 dev server 실행

```
npm run dev &
sleep 5
```

## 9.3 probe 실행

```
npx playwright test tests/architecture/_probe-stream-pipeline.spec.ts \
 --reporter=line | tee /tmp/probe_stream.log

npx playwright test tests/architecture/_probe-detect-flow.spec.ts \
 --reporter=line | tee /tmp/probe_detect.log

npx playwright test tests/architecture/_probe-gimbal-tracking.spec.ts \
 --reporter=line | tee /tmp/probe_gimbal.log

npx playwright test tests/architecture/_probe-phenotyping.spec.ts \
 --reporter=line | tee /tmp/probe_phenotyping.log

npx playwright test tests/architecture/logger-enforcement.spec.ts \
 --reporter=line | tee /tmp/logger_enforcement.log
```

## 9.4 결과 정리

각 로그 파일을 시험 결과 보고서( `10_Test_Report.pdf` )의 부록으로 첨부.

## 9.5 종료

```
kill %1 # dev server 종료
```